TP 3 — Kubernetes (GKE), Apache Kafka & Pipeline CI/CD

Cours 3 | Développer pour le Cloud | YNOV Campus Montpellier — Master 2

Date : 28/04/2026 | **Durée TP :** 3h30 | **Plateforme :** Google Cloud Platform

---

> ***Contexte entreprise — LogiStream***
>
> *LogiStream est une startup B2B qui fournit une plateforme de suivi de livraisons en temps réel à des transporteurs routiers. Leurs chauffeurs envoient leur position GPS toutes les 10 secondes depuis une application mobile. Avec 800 camions actifs simultanément, ça représente **80 événements/seconde** en continu. L'architecture actuelle (un serveur Node.js + PostgreSQL) ne tient plus la charge : les positions arrivent avec 45 secondes de retard aux heures de pointe, rendant le suivi client inutilisable. La solution retenue : déployer les microservices sur **GKE** et implémenter **Apache Kafka** comme backplane de streaming pour absorber les pics de charge. Vous êtes le/la Cloud & Streaming Engineer en charge de cette migration.*

> ***Prérequis validés (Cours 2) :***
>
> - *Application `tp2-app` déployée sur Cloud Run et pushée dans Artifact Registry*
> - `kubectl` *installé (* `gcloud components install kubectl` *)*
> - *Docker fonctionnel en local*

Objectifs de ce TP :

- Déployer et opérer des microservices sur GKE (Deployment, Service, ConfigMap, Secret, HPA)
- Installer Apache Kafka sur Kubernetes via l'opérateur Strimzi
- Développer des producers et consumers Kafka en Node.js avec KafkaJS
- Créer un pipeline CI/CD GitHub Actions complet (test → build → push → deploy)
- Superviser le cluster Kafka avec les métriques Cloud Monitoring

Livrables attendus :

- [ ] Microservices déployés sur GKE avec HPA fonctionnel
- [ ] Cluster Kafka 3 brokers opérationnel sur Kubernetes
- [ ] Producer qui envoie des positions GPS et Consumer qui les traite en temps réel
- [ ] Pipeline GitHub Actions avec 3 jobs réussis
- [ ] `README.md` avec diagramme de l'architecture LogiStream

Partie 1 — Déploiement des microservices LogiStream sur GKE

> *On déploie d'abord les microservices métier : un **API Gateway** (point d'entrée HTTP) et un **Tracker Service** (traitement des positions GPS). Ces services communiqueront ensuite via Kafka.*

1.1 — Créer le cluster GKE

```
# Créer un cluster GKE Autopilot (nodes gérés automatiquement par GCP)
gcloud container clusters create-auto logistream-cluster \
  --region=europe-west9 \
  --project=$(gcloud config get-value project)

# Configurer kubectl pour pointer sur ce cluster
gcloud container clusters get-credentials logistream-cluster \
  --region=_______

# Vérifier la connexion au cluster
kubectl _______   # Commande pour lister les nodes

# Créer un namespace dédié à LogiStream (bonne pratique : 1 namespace par
application)
kubectl create namespace logistream

# Vérifier la création du namespace
kubectl get namespaces
```

1.2 — ConfigMap : configuration centralisée des services

Créez `k8s/configmap.yaml` :

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: logistream-config
  namespace: _______   # logistream
data:
  # Configuration applicative (non sensible)
  APP_ENV: "production"
  LOG_LEVEL: "info"
  KAFKA_BOOTSTRAP_SERVERS: "kafka-cluster-kafka-
bootstrap.logistream.svc.cluster.local:9092"
  KAFKA_TOPIC_POSITIONS: "truck-positions"
  KAFKA_TOPIC_ALERTS: "delivery-alerts"
  GPS_UPDATE_INTERVAL_MS: "10000"
  # TODO : ajouter MAX_TRUCKS avec la valeur "1000"
  _______: "_______"
```

1.3 — Secret : credentials de base de données

Créez `k8s/secret.yaml` :

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: logistream-secrets
  namespace: logistream
```

```yaml
type: Opaque
stringData:
  # stringData : Kubernetes encode automatiquement en base64
  DB_URL: "postgresql://logistream:motdepasse@cloud-sql-proxy:5432/logistream_prod"
  MAPS_API_KEY: "AIza-demo-logistream-maps-key-tp3"
  JWT_SECRET: "logistream-jwt-secret-2026-production"
```

Question : Dans Kubernetes, un Secret de type `Opaque` encode les valeurs en base64 mais ne les chiffre pas au repos par défaut. Quelle configuration GKE permet de chiffrer les Secrets ETCD avec une clé Cloud KMS ? Pourquoi est-ce indispensable en production ?

```
Réponse :
```

1.4 — Deployment : l'API Gateway LogiStream

Créez `k8s/api-gateway.yaml` :

```yaml
apiVersion: apps/v1
kind: _______   # Deployment
metadata:
  name: api-gateway
  namespace: logistream
  labels:
    app: api-gateway
    team: backend
spec:
  replicas: _______   # 2 réplicas pour la haute disponibilité
  selector:
    matchLabels:
      app: api-gateway

  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0   # Zéro interruption pendant les mises à jour

  template:
    metadata:
      labels:
        app: api-gateway
    spec:
      containers:
        - name: api-gateway
          image: europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/tp2-app:v1
          ports:
            - containerPort: 8080

          # Variables depuis le ConfigMap (toutes les clés deviennent des variables
d'env)
          envFrom:
```

```yaml
            - configMapRef:
                name: _______   # logistream-config

          # Variable individuelle depuis le Secret
          env:
            - name: JWT_SECRET
              valueFrom:
                secretKeyRef:
                  name: logistream-secrets
                  key: _______   # JWT_SECRET

          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"

          readinessProbe:
            httpGet:
              path: /health
              port: _______   # 8080
            initialDelaySeconds: 5
            periodSeconds: 10

          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: api-gateway-svc
  namespace: logistream
spec:
  selector:
    app: _______   # api-gateway
  ports:
    - port: 80
      targetPort: 8080
  type: LoadBalancer
```

1.5 — Deployment : le Tracker Service

Créez `k8s/tracker-service.yaml` :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tracker-service
  namespace: logistream
  labels:
    app: tracker-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: tracker-service
  template:
    metadata:
      labels:
        app: tracker-service
    spec:
      containers:
        - name: tracker-service
          image: europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/tp2-app:v1
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: logistream-config
          env:
            - name: DB_URL
              valueFrom:
                secretKeyRef:
                  name: logistream-secrets
                  key: _______    # DB_URL
            - name: SERVICE_ROLE
              value: "tracker"
          resources:
            requests:
              cpu: "200m"
              memory: "256Mi"
            limits:
              cpu: "1000m"
              memory: "1Gi"
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: tracker-service-svc
```

```yaml
  namespace: logistream
spec:
  selector:
    app: tracker-service
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP   # Pas d'exposition externe : ce service est appelé par l'API
Gateway
```

1.6 — HorizontalPodAutoscaler : absorber les pics de charge

> *LogiStream doit tenir les pics de 08h00 le lundi (tous les camions démarrent en même temps). Sans HPA, les pods saturent. Avec HPA, Kubernetes ajoute automatiquement des réplicas quand le CPU dépasse le seuil.*

Créez `k8s/hpa.yaml` :

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: tracker-hpa
  namespace: logistream
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: _______   # Deployment
    name: tracker-service

  minReplicas: 2
  maxReplicas: _______   # 10 pods maximum

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: _______   # Scaler si CPU > 60%
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 70
```

1.7 — Appliquer et tester

```
# Appliquer tous les manifests
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secret.yaml
kubectl apply -f k8s/api-gateway.yaml
kubectl apply -f k8s/tracker-service.yaml
kubectl apply -f k8s/_______   # hpa.yaml

# Vérifier que tous les pods sont Running (READY 1/1)
kubectl get pods -n logistream -w

# Récupérer l'IP publique de l'API Gateway
kubectl get service api-gateway-svc -n logistream
GATEWAY_IP=$(kubectl get service api-gateway-svc -n logistream \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

# Tester l'application
curl http://${GATEWAY_IP}/health
curl http://${GATEWAY_IP}/

# Simuler une charge CPU pour observer le HPA en action
kubectl run load-test \
  --image=busybox:latest \
  --restart=Never \
  -n logistream \
  -- /bin/sh -c "while true; do wget -qO- http://tracker-service-svc/; done"

# Observer le HPA scaler (attendre 1-2 minutes)
kubectl get hpa tracker-hpa -n logistream -w

# Arrêter le test de charge
kubectl delete pod load-test -n logistream
```

Question : Le tracker-service reçoit 80 événements GPS/seconde en pointe. Le HPA est configuré avec `minReplicas: 2` et `maxReplicas: 10`. Si chaque pod peut traiter 20 événements/seconde à 60% CPU, combien de réplicas le HPA maintiendra-t-il en régime de pointe ?

```
Réponse :
```

Partie 2 — Apache Kafka sur Kubernetes avec Strimzi

> *Strimzi est l'opérateur Kubernetes officiel pour Apache Kafka. Il gère le cycle de vie complet du cluster Kafka (déploiement, configuration, mise à jour, scaling) via des Custom Resource Definitions (CRD) Kubernetes.*

2.1 — Installer l'opérateur Strimzi

```
# Créer le namespace pour Kafka
kubectl create namespace kafka
```

```
# Installer Strimzi via le fichier d'installation officiel (version 0.39)
kubectl apply -f https://strimzi.io/install/latest?namespace=kafka \
  -n kafka

# Vérifier que l'opérateur Strimzi est Running
kubectl get pods -n kafka -w
# Attendre : strimzi-cluster-operator-xxx   Running

# Lister les Custom Resource Definitions (CRD) installées par Strimzi
kubectl get crds | grep kafka
# Résultat attendu :
# kafkaclusters.kafka.strimzi.io
# kafkatopics.kafka.strimzi.io
# kafkausers.kafka.strimzi.io
# kafkaconnectors.kafka.strimzi.io
```

2.2 — Créer le cluster Kafka (KRaft mode — sans ZooKeeper)

> *Le mode **KRaft** (Kafka Raft) est le mode natif de Kafka depuis la version 3.3 : il n'a plus besoin de ZooKeeper, ce qui simplifie le déploiement et réduit les coûts.*

Créez `kafka/cluster.yaml` :

```yaml
apiVersion: kafka.strimzi.io/v1beta2
kind: Kafka
metadata:
  name: logistream-kafka
  namespace: logistream
spec:
  kafka:
    version: 3.7.0
    replicas: _______   # 3 brokers pour la haute disponibilité
    listeners:
      # Listener interne (pour les pods dans le cluster Kubernetes)
      - name: plain
        port: 9092
        type: internal
        tls: false
      # Listener externe (pour les clients hors du cluster)
      - name: external
        port: 9094
        type: _______   # loadbalancer (expose via un Load Balancer GCP)
        tls: false
    config:
      # Nombre de partitions par défaut pour les nouveaux topics
      num.partitions: 3
      # Facteur de réplication par défaut
      default.replication.factor: _______   # 3
      # Nombre minimum de réplicas IN-SYNC pour écrire (sécurité)
      min.insync.replicas: 2
      # Rétention des messages : 7 jours
```

```
      log.retention.hours: 168
      # Taille max d'un message GPS : 10 KB (assez pour les coordonnées +
métadonnées)
      message.max.bytes: 10240

    resources:
      requests:
        memory: "1Gi"
        cpu: "500m"
      limits:
        memory: "2Gi"
        cpu: "1"

    storage:
      type: persistent-claim
      size: _______    # 20Gi par broker
      deleteClaim: true   # Supprimer les volumes quand le cluster est supprimé

  # Mode KRaft : pas de ZooKeeper
  zookeeper:
    replicas: 0   # Désactivé (KRaft mode)

  entityOperator:
    topicOperator: {}   # Gère les KafkaTopic CRD
    userOperator: {}    # Gère les KafkaUser CRD
```

```
# Déployer le cluster Kafka
kubectl apply -f kafka/cluster.yaml -n logistream

# Suivre la progression (le cluster prend 3-5 minutes à démarrer)
kubectl get kafka logistream-kafka -n logistream -w

# Résultat attendu quand prêt :
# NAME               DESIRED KAFKA REPLICAS   READY   ...
# logistream-kafka   3                        True

# Vérifier les pods Kafka (3 brokers)
kubectl get pods -n logistream -l strimzi.io/cluster=logistream-kafka

# Récupérer l'adresse du bootstrap server (pour les clients)
kubectl get kafka logistream-kafka -n logistream \
  -o jsonpath='{.status.listeners[0].bootstrapServers}'
```

2.3 — Créer les topics Kafka via KafkaTopic CRD

> *En mode GitOps/Kubernetes, on ne crée pas les topics avec `kafka-topics.sh` en ligne de commande. On utilise la CRD `KafkaTopic` : l'opérateur Strimzi crée et maintient le topic automatiquement.*

Créez `kafka/topics.yaml` :

```yaml
# Topic 1 : Positions GPS des camions (flux principal)
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: truck-positions
  namespace: logistream
  labels:
    strimzi.io/cluster: logistream-kafka   # Lie ce topic au cluster ci-dessus
spec:
  partitions: _______   # 6 partitions (une par zone géographique — Nord, Sud, Est,
Ouest, Centre, IDF)
  replicas: 3
  config:
    # Rétention : 24h (les positions ne sont utiles qu'en temps réel)
    retention.ms: "86400000"
    # Compaction : garder la dernière position par camion (clé = truck_id)
    cleanup.policy: "_______"   # delete (pas de compaction ici, on garde
l'historique)
---
# Topic 2 : Alertes de livraison (retards, incidents)
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: delivery-alerts
  namespace: logistream
  labels:
    strimzi.io/cluster: logistream-kafka
spec:
  partitions: 3
  replicas: _______   # 3 réplicas
  config:
    # Rétention : 7 jours (les alertes doivent être auditables)
    retention.ms: "604800000"
    cleanup.policy: "delete"
---
# Topic 3 : Événements de livraison (chargement, déchargement, signature)
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: delivery-events
  namespace: logistream
  labels:
    strimzi.io/cluster: logistream-kafka
spec:
  partitions: 3
  replicas: 3
  config:
    retention.ms: "2592000000"   # 30 jours
    cleanup.policy: "delete"
```

```
# Appliquer les topics
kubectl apply -f kafka/topics.yaml

# Vérifier la création des topics
kubectl get kafkatopics -n logistream

# Résultat attendu :
# NAME               CLUSTER            PARTITIONS   REPLICATION FACTOR   READY
# truck-positions    logistream-kafka   6            3                    True
# delivery-alerts    logistream-kafka   3            3                    True
# delivery-events    logistream-kafka   3            3                    True

# Vérifier via l'outil CLI Kafka (depuis un pod temporaire)
kubectl run kafka-cli \
  --image=quay.io/strimzi/kafka:0.39.0-kafka-3.7.0 \
  --restart=Never \
  -n logistream \
  -- /bin/bash -c "bin/kafka-topics.sh \
    --bootstrap-server logistream-kafka-kafka-bootstrap:9092 \
    --list"

kubectl logs kafka-cli -n logistream
kubectl delete pod kafka-cli -n logistream
```

2.4 — Producer Kafka : simuler les positions GPS des camions

Créez `producer/gps-producer.js` :

```
const { Kafka, Partitioners } = require('kafkajs');

// Configuration du client Kafka
const kafka = new Kafka({
  clientId: 'logistream-gps-producer',
  brokers: [process.env.KAFKA_BOOTSTRAP_SERVERS || 'localhost:9092'],
  // Retry automatique avec backoff exponentiel
  retry: {
    initialRetryTime: 100,
    retries: 8,
  },
});

const producer = kafka.producer({
  createPartitioner: Partitioners.LegacyPartitioner,
});

// Simuler la flotte de camions LogiStream
const TRUCKS = [
  { id: 'TRK-001', driver: 'Martin Dupont', route: 'Paris-Lyon' },
  { id: 'TRK-002', driver: 'Sophie Laurent', route: 'Lyon-Marseille' },
  { id: 'TRK-003', driver: 'Jean Moreau', route: 'Bordeaux-Paris' },
```

```
];

function generateGPSPosition(truck) {
  // Simuler un mouvement progressif sur la route
  const basePositions = {
    'Paris-Lyon':       { lat: 48.8566, lng: 2.3522 },
    'Lyon-Marseille':   { lat: 45.7640, lng: 4.8357 },
    'Bordeaux-Paris':   { lat: 44.8378, lng: -0.5792 },
  };

  const base = basePositions[truck.route];
  return {
    truck_id: truck.id,
    driver: truck.driver,
    route: truck.route,
    // Variation aléatoire ±0.1 degré pour simuler le mouvement
    latitude: base.lat + (Math.random() - 0.5) * 0.1,
    longitude: base.lng + (Math.random() - 0.5) * 0.1,
    speed_kmh: Math.floor(Math.random() * 40) + 70,   // 70-110 km/h
    fuel_level: Math.floor(Math.random() * 60) + 20,   // 20-80%
    timestamp: new Date().toISOString(),
    event_type: 'GPS_UPDATE',
  };
}

async function startProducing() {
  await producer.connect();
  console.log('Producer Kafka connecté — début de l\'envoi des positions GPS');

  let messageCount = 0;

  // Envoyer une position GPS pour chaque camion toutes les 10 secondes
  setInterval(async () => {
    const messages = TRUCKS.map(truck => {
      const position = generateGPSPosition(truck);
      return {
        // La clé de partition = truck_id : toutes les positions d'un camion
        // vont dans la même partition → ordre garanti par camion
        key: _______,   // truck.id
        value: JSON.stringify(position),
        headers: {
          'event-type': 'gps-position',
          'source': 'mobile-app',
        },
      };
    });

    try {
      await producer.send({
        topic: '_______',   // 'truck-positions'
        messages: messages,
      });
```

```
      messageCount += messages.length;
      console.log(`[${new Date().toISOString()}] ${messages.length} positions
envoyées (total: ${messageCount})`);
    } catch (err) {
      console.error('Erreur d\'envoi Kafka :', err.message);
    }
  }, 10000);   // Toutes les 10 secondes

  // Envoyer une alerte simulée toutes les 30 secondes
  setInterval(async () => {
    const truck = TRUCKS[Math.floor(Math.random() * TRUCKS.length)];
    const alert = {
      truck_id: truck.id,
      alert_type: 'DELIVERY_DELAY',
      severity: 'WARNING',
      message: `Camion ${truck.id} en retard de 15 minutes sur la route
${truck.route}`,
      estimated_delay_minutes: Math.floor(Math.random() * 30) + 5,
      timestamp: new Date().toISOString(),
    };

    await producer.send({
      topic: 'delivery-alerts',
      messages: [{ key: truck.id, value: JSON.stringify(alert) }],
    });

    console.log(`[ALERTE] ${alert.message}`);
  }, 30000);
}

// Gestion propre de l'arrêt (SIGTERM pour Kubernetes)
process.on('SIGTERM', async () => {
  console.log('SIGTERM reçu — arrêt propre du producer');
  await producer.disconnect();
  process.exit(0);
});

startProducing().catch(err => {
  console.error('Erreur fatale :', err);
  process.exit(1);
});
```

Créez `producer/package.json` :

```
{
  "name": "logistream-gps-producer",
  "version": "1.0.0",
  "description": "Simulateur GPS pour la flotte LogiStream",
  "main": "gps-producer.js",
  "scripts": {
```

```
    "start": "node gps-producer.js"
  },
  "dependencies": {
    "kafkajs": "^2.2.4"
  },
  "engines": { "node": "20" }
}
```

2.5 — Consumer Kafka : le Tracker Service traite les positions

Créez `consumer/tracker-consumer.js` :

```
const { Kafka } = require('kafkajs');

const kafka = new Kafka({
  clientId: 'logistream-tracker-consumer',
  brokers: [process.env.KAFKA_BOOTSTRAP_SERVERS || 'localhost:9092'],
});

// Consumer Group : si plusieurs instances du tracker-service tournent,
// Kafka distribue les partitions entre elles automatiquement
const consumer = kafka.consumer({
  groupId: '_______',   // 'tracker-service-group'
});

// Simulation du stockage en base de données
const truckPositions = new Map();

async function startConsuming() {
  await consumer.connect();
  console.log('Consumer Kafka connecté');

  // S'abonner aux topics pertinents
  await consumer.subscribe({
    topics: ['_______', 'delivery-alerts'],   // ['truck-positions', ...]
    fromBeginning: false,   // Partir du dernier offset (pas de relecture
historique)
  });

  await consumer.run({
    // Nombre de messages traités en parallèle par partition
    partitionsConsumedConcurrently: 3,

    eachMessage: async ({ topic, partition, message }) => {
      const key = message.key?.toString();
      const value = JSON.parse(message.value.toString());
      const offset = message.offset;

      if (topic === 'truck-positions') {
        // Mettre à jour la position du camion en mémoire (et en DB en prod)
        truckPositions.set(key, {
```

```
          ...value,
          last_seen: new Date().toISOString(),
          processed_at: Date.now(),
        });

        console.log(`[POSITION] ${value.truck_id} |
${value.latitude.toFixed(4)},${value.longitude.toFixed(4)} | ${value.speed_kmh} km/h
| Partition ${partition} | Offset ${offset}`);

        // Détecter les camions à l'arrêt (vitesse < 5 km/h depuis plus de 30 min)
        if (value.speed_kmh < 5) {
          await publishAlert(value.truck_id, 'TRUCK_STOPPED', value);
        }

        // Détecter le carburant bas (< 20%)
        if (value.fuel_level < _______) {   // 20
          await publishAlert(value.truck_id, 'LOW_FUEL', value);
        }

      } else if (topic === 'delivery-alerts') {
        // Traiter les alertes : notifier les dispatchers
        console.log(`[ALERTE] ${value.alert_type} | ${value.truck_id} |
${value.message}`);

        // En production : envoyer un email/SMS au dispatcher
        await notifyDispatcher(value);
      }
    },
  });
}

async function publishAlert(truckId, alertType, context) {
  // En production : produire vers le topic delivery-alerts
  console.log(`[DÉTECTION] ${alertType} pour ${truckId}`);
}

async function notifyDispatcher(alert) {
  // En production : appel à l'API de notification (email/SMS)
  console.log(`[NOTIFICATION] Dispatcher alerté : ${alert.alert_type}`);
}

// Arrêt propre : commit des offsets avant de quitter
process.on('SIGTERM', async () => {
  console.log('SIGTERM reçu — commit des offsets en cours');
  await consumer.disconnect();
  process.exit(0);
});

startConsuming().catch(err => {
  console.error('Erreur consumer :', err);
```

```
  process.exit(1);
});
```

Question : Dans KafkaJS, le paramètre `groupId` permet à plusieurs instances du `tracker-service` de former un **Consumer Group**. Expliquez comment Kafka distribue les partitions du topic `truck-positions` (6 partitions) entre 3 instances du consumer. Que se passe-t-il si on ajoute une 4e instance ?

```
Réponse :
```

2.6 — Déployer le Producer et Consumer sur GKE

Créez le Dockerfile pour le producer (`producer/Dockerfile`) :

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
USER node
CMD ["node", "gps-producer.js"]
```

```
PROJECT_ID=$(gcloud config get-value project)

# Builder et pusher les images
docker build -t europe-west9-docker.pkg.dev/${PROJECT_ID}/tp2-registry/gps-producer:v1 ./producer/
docker push europe-west9-docker.pkg.dev/${PROJECT_ID}/tp2-registry/gps-producer:v1

docker build -t europe-west9-docker.pkg.dev/${PROJECT_ID}/tp2-registry/tracker-consumer:v1 ./consumer/
docker push europe-west9-docker.pkg.dev/${PROJECT_ID}/tp2-registry/tracker-consumer:v1
```

Créez `k8s/kafka-apps.yaml` :

```
# Deployment du GPS Producer (simule la flotte de camions)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gps-producer
  namespace: logistream
spec:
  replicas: 1   # 1 seul producer suffit (il simule toute la flotte)
  selector:
    matchLabels:
      app: gps-producer
  template:
    metadata:
      labels:
```

```yaml
        app: gps-producer
    spec:
      containers:
        - name: gps-producer
          image: europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/gps-
producer:v1
          envFrom:
            - configMapRef:
                name: logistream-config
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
---
# Deployment du Tracker Consumer (traite les positions GPS)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tracker-consumer
  namespace: logistream
spec:
  # 3 réplicas = 3 instances dans le même Consumer Group
  # Kafka distribue les 6 partitions : 2 partitions par instance
  replicas: _______   # 3
  selector:
    matchLabels:
      app: tracker-consumer
  template:
    metadata:
      labels:
        app: tracker-consumer
    spec:
      containers:
        - name: tracker-consumer
          image: europe-west9-docker.pkg.dev/[PROJECT_ID]/tp2-registry/tracker-
consumer:v1
          envFrom:
            - configMapRef:
                name: logistream-config
          resources:
            requests:
              cpu: "200m"
              memory: "256Mi"
```

```bash
kubectl apply -f k8s/kafka-apps.yaml

# Observer les logs du producer (positions GPS envoyées)
kubectl logs -f deployment/gps-producer -n logistream

# Observer les logs du consumer (positions GPS traitées)
kubectl logs -f deployment/tracker-consumer -n logistream
```

```
# Vérifier les offsets consommés (lag = retard du consumer)
kubectl run kafka-consumer-groups \
  --image=quay.io/strimzi/kafka:0.39.0-kafka-3.7.0 \
  --restart=Never \
  -n logistream \
  -- /bin/bash -c "bin/kafka-consumer-groups.sh \
    --bootstrap-server logistream-kafka-kafka-bootstrap:9092 \
    --describe \
    --group tracker-service-group"

kubectl logs kafka-consumer-groups -n logistream
kubectl delete pod kafka-consumer-groups -n logistream
```

Partie 3 — Pipeline CI/CD avec GitHub Actions

> *LogiStream veut automatiser le déploiement : chaque push sur `main` doit déclencher les tests, builder les nouvelles images et déployer automatiquement sur GKE, sans intervention manuelle.*

3.1 — Préparer le repository GitHub

```
git init
git add .
git commit -m "feat: initial commit LogiStream GKE + Kafka"

gh repo create logistream-cloud --public --push --source=.
```

3.2 — Secrets GitHub requis

Dans **Settings → Secrets and variables → Actions**, créer :

Secret	Description
`GCP_PROJECT_ID`	ID de votre projet GCP
`GCP_SA_KEY`	Clé JSON du Service Account CI/CD
`GKE_CLUSTER`	`logistream-cluster`
`GKE_REGION`	`europe-west9`

```
PROJECT_ID=$(gcloud config get-value project)

# Créer le Service Account pour GitHub Actions
gcloud iam service-accounts create logistream-cicd-sa \
  --display-name="LogiStream CI/CD"

SA_EMAIL="logistream-cicd-sa@${PROJECT_ID}.iam.gserviceaccount.com"

# Permissions nécessaires
```

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/artifactregistry.writer"

gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/_______"   # container.developer

# Générer la clé JSON
gcloud iam service-accounts keys create cicd-key.json --iam-account=${SA_EMAIL}
cat cicd-key.json   # Copier dans GitHub Secret GCP_SA_KEY
rm cicd-key.json    # Supprimer immédiatement après copie
```

3.3 — Pipeline GitHub Actions complet

Créez `.github/workflows/logistream-deploy.yml` :

```yaml
name: LogiStream — CI/CD Pipeline

on:
  push:
    branches: [_______]   # main
  pull_request:
    branches: [main]

env:
  PROJECT_ID: ${{ secrets.GCP_PROJECT_ID }}
  GKE_CLUSTER: ${{ secrets.GKE_CLUSTER }}
  GKE_REGION: ${{ secrets.GKE_REGION }}
  REGISTRY: europe-west9-docker.pkg.dev

jobs:
  # ============================================================
  # Job 1 : Tests (s'exécute sur chaque PR et chaque push)
  # ============================================================
  test:
    name: Tests & Lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Installer les dépendances
        run: npm ci

      - name: Tests unitaires
        run: npm test
```

```yaml
      - name: Lint du code
        run: npm run lint || echo "Lint non configuré — skip"

      - name: Valider les manifests Kubernetes
        run: |
          # Installer kubeval pour valider les YAML Kubernetes
          curl -sL
https://github.com/instrumenta/kubeval/releases/latest/download/kubeval-linux-
amd64.tar.gz | tar xz
          ./kubeval k8s/*.yaml --ignore-missing-schemas

  # ============================================================
  # Job 2 : Build & Push (uniquement sur push main)
  # ============================================================
  build-push:
    name: Build & Push Images
    runs-on: ubuntu-latest
    needs: _______   # test
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'

    outputs:
      producer-tag: ${{ steps.tags.outputs.producer-tag }}
      consumer-tag: ${{ steps.tags.outputs.consumer-tag }}

    steps:
      - uses: actions/checkout@v4

      - name: Authentification GCP
        uses: google-github-actions/auth@v2
        with:
          credentials_json: ${{ secrets.GCP_SA_KEY }}

      - uses: google-github-actions/setup-gcloud@v2

      - name: Configurer Docker
        run: gcloud auth configure-docker ${REGISTRY} --quiet

      - name: Générer les tags d'image
        id: tags
        run: |
          SHA="${{ github.sha }}"
          echo "producer-tag=${REGISTRY}/${PROJECT_ID}/tp2-registry/gps-
producer:${SHA}" >> $GITHUB_OUTPUT
          echo "consumer-tag=${REGISTRY}/${PROJECT_ID}/tp2-registry/tracker-
consumer:${SHA}" >> $GITHUB_OUTPUT

      - name: Build & Push GPS Producer
        run: |
          docker build -t ${{ steps.tags.outputs.producer-tag }} ./producer/
          docker push ${{ steps.tags.outputs.producer-tag }}
```

```yaml
      - name: Build & Push Tracker Consumer
        run: |
          docker build -t ${{ steps.tags.outputs.consumer-tag }} ./consumer/
          docker push _______   # ${{ steps.tags.outputs.consumer-tag }}

  # ==========================================================
  # Job 3 : Deploy sur GKE
  # ==========================================================
  deploy:
    name: Deploy sur GKE
    runs-on: ubuntu-latest
    needs: _______   # build-push

    steps:
      - uses: actions/checkout@v4

      - name: Authentification GCP
        uses: google-github-actions/auth@v2
        with:
          credentials_json: ${{ secrets.GCP_SA_KEY }}

      - uses: google-github-actions/setup-gcloud@v2
        with:
          install_components: 'kubectl'

      - name: Configurer kubectl
        run: |
          gcloud container clusters get-credentials ${GKE_CLUSTER} \
            --region=${GKE_REGION} --project=${PROJECT_ID}

      - name: Mettre à jour le GPS Producer
        run: |
          kubectl set image deployment/gps-producer \
            gps-producer=${{ needs.build-push.outputs.producer-tag }} \
            -n logistream

      - name: Mettre à jour le Tracker Consumer
        run: |
          kubectl set image deployment/tracker-consumer \
            tracker-consumer=${{ needs.build-push.outputs.consumer-tag }} \
            -n logistream

      - name: Attendre les déploiements
        run: |
          kubectl rollout status deployment/gps-producer -n logistream --timeout=5m
          kubectl rollout status deployment/tracker-consumer -n logistream --
timeout=5m

      - name: Vérification post-déploiement
        run: |
          kubectl get pods -n logistream
```

```
          kubectl get kafkatopics -n logistream
          echo "Déploiement LogiStream réussi ✓"
```

```
# Déclencher le pipeline
git add .
git commit -m "feat: add Kafka streaming and CI/CD pipeline"
git push origin main

# Surveiller l'exécution
gh run list
gh run watch
```

Question : Le pipeline valide les manifests Kubernetes avec `kubeval`. Quel autre outil de validation vous permettrait de vérifier que les manifests respectent les règles de sécurité de l'entreprise (ex: tout container doit avoir des `resources.limits`, tout pod doit avoir une `readinessProbe`) ?

```
Réponse :
```

Partie 4 — Observabilité du cluster Kafka et des microservices

4.1 — Métriques Kafka avec Cloud Monitoring

```
# Vérifier les métriques disponibles pour Kafka
# Strimzi expose les métriques JMX via Prometheus
# Dans GKE, Cloud Monitoring peut les scraper

# Observer les métriques des pods Kafka directement
kubectl top pods -n logistream -l strimzi.io/cluster=logistream-kafka

# Vérifier le lag du consumer group (retard dans la consommation)
kubectl run kafka-lag-check \
  --image=quay.io/strimzi/kafka:0.39.0-kafka-3.7.0 \
  --restart=Never \
  -n logistream \
  -- /bin/bash -c "
    bin/kafka-consumer-groups.sh \
      --bootstrap-server logistream-kafka-kafka-bootstrap:9092 \
      --describe \
      --group tracker-service-group 2>/dev/null;
    echo '---';
    bin/kafka-topics.sh \
      --bootstrap-server logistream-kafka-kafka-bootstrap:9092 \
      --describe \
      --topic truck-positions
  "

sleep 15
```

```
kubectl logs kafka-lag-check -n logistream
kubectl delete pod kafka-lag-check -n logistream
```

4.2 — Logs structurés et requêtes Cloud Logging

```
# Logs du producer (positions GPS envoyées)
gcloud logging read \
  'resource.type="k8s_container"
   AND resource.labels.namespace_name="logistream"
   AND resource.labels.container_name="gps-producer"' \
  --limit=20 \
  --format="table(timestamp,jsonPayload.message)"

# Logs du consumer filtré sur les alertes seulement
gcloud logging read \
  'resource.type="k8s_container"
   AND resource.labels.namespace_name="logistream"
   AND resource.labels.container_name="tracker-consumer"
   AND textPayload:"[ALERTE]"' \
  --limit=10

# Logs d'erreur du cluster Kafka
gcloud logging read \
  'resource.type="k8s_container"
   AND resource.labels.namespace_name="logistream"
   AND resource.labels.container_name:"kafka"
   AND severity>=_______' \
  --limit=20
# Niveau de sévérité : ERROR
```

4.3 — Créer une alerte sur le Consumer Lag

> *Un consumer lag élevé sur `truck-positions` signifie que le Tracker Service n'arrive pas à traiter les positions assez vite : les chauffeurs verront des positions en retard dans l'interface. C'est le problème exact que LogiStream a eu avec son ancienne architecture.*

Dans la console GCP (**Monitoring → Alerting → Create Policy**) :

```
Étape 1 — Métrique :
  Type      : Kubernetes Container
  Métrique  : kubernetes.io/container/memory/request_utilization
  Filtre    : namespace_name = "logistream", container_name = "tracker-consumer"

Étape 2 — Condition :
  Seuil     : > 80%
  Durée     : 5 minutes

Étape 3 — Notification :
```

```
  Canal     : Votre email
  Nom       : "LogiStream Tracker Consumer — Mémoire haute"
```

Notez les métriques observées au repos :

Métrique	Valeur observée
CPU moyen tracker-consumer	_____ %
RAM moyenne tracker-consumer	_____ Mi
Pods HPA actifs au repos	_____

Question : Vous observez que le consumer lag sur `truck-positions` monte progressivement au cours du temps. Vous avez 3 instances du tracker-consumer (3 réplicas). Quelles sont les 3 actions à envisager dans l'ordre pour résoudre ce problème ?

```
Réponse :
```

Nettoyage Final — IMPORTANT

```
# 1. Supprimer les ressources applicatives Kubernetes
kubectl delete -f k8s/ -n logistream

# 2. Supprimer le cluster Kafka (et les volumes persistants)
kubectl delete kafka logistream-kafka -n logistream
kubectl delete kafkatopics --all -n logistream

# 3. Désinstaller Strimzi
kubectl delete -f https://strimzi.io/install/latest?namespace=kafka -n kafka

# 4. Supprimer le namespace logistream
kubectl delete namespace logistream kafka

# 5. Supprimer le cluster GKE (IRRÉVERSIBLE)
gcloud container clusters delete logistream-cluster --region=europe-west9

# 6. Vérification finale
gcloud container clusters list
kubectl get namespaces
```

Récapitulatif — Compétences validées

- [ ] Kubernetes : Deployment, Service, ConfigMap, Secret, HPA
- [ ] Apache Kafka (Strimzi) : cluster KRaft, KafkaTopic CRD, Consumer Groups
- [ ] KafkaJS : Producer avec partitioning par clé, Consumer avec groupId
- [ ] CI/CD : GitHub Actions (test → build → push → deploy sur GKE)
- [ ] Observabilité : Cloud Logging, Cloud Monitoring, consumer lag

Livrables finaux à remettre

- [ ] Repository Git public avec le code complet (k8s/ + kafka/ + producer/ + consumer/ + .github/)
- [ ] Captures d'écran des logs producer ET consumer fonctionnels simultanément
- [ ] Résultat de la commande `kafka-consumer-groups.sh --describe` (screenshot)
- [ ] Pipeline GitHub Actions avec 3 jobs verts (screenshot)
- [ ] `README.md` avec diagramme d'architecture LogiStream (camions → GPS → Kafka → GKE → dashboard)